

Smart Group Trip Planner API

3-Day Python + Flask + SQLite Challenge

Duration	3 days
Type	Individual assignment
Technology	Python, Flask, SQLite
Submission	Git repository
Repository pattern	{firstname}_{lastname}_trip_planner_batch_12

Goal: build a small but robust REST API where correctness of business rules matters more than the number of endpoints.

1. Assignment Objective

Build a REST API for managing group trips, travelers, expenses, and trip lifecycle rules. The system must persist data in SQLite and must be runnable by an evaluator from a fresh clone with a predictable command.

- Practice Python fundamentals through real application logic.
- Build REST endpoints with Flask.
- Persist relational data using SQLite.
- Apply validation, state transitions, calculations, and conflict detection.
- Use appropriate HTTP methods, JSON responses, and status codes.
- Demonstrate clean Git usage, naming conventions, documentation, and reproducible setup.

2. Problem Scenario

A travel organization needs a backend service to manage group trips. A trip has a destination, date range, budget, capacity, travelers, expenses, and lifecycle status. The system must prevent invalid operations such as overbooking, duplicate participation, overlapping trips for the same traveler, overspending, and invalid status transitions.

3. Required Technology and Scope

- Python 3 and Flask are mandatory.
- SQLite is mandatory for persistent storage.
- A Flask-compatible ORM such as Flask-SQLAlchemy/SQLAlchemy may be used.
- No frontend is required.
- Authentication/authorization is outside the scope.
- Deployment is outside the scope.
- Database migration tooling is optional unless you choose to use it.
- The solution must not depend on manually creating tables or editing local configuration before first run.

4. Repository and Naming Convention

Repository name:

```
{firstname}_{lastname}_trip_planner_batch_12
```

Example:

```
muhammad_rajib_trip_planner_batch_12
```

- Use lowercase repository names.
- Use underscores between words.
- Python variables and functions: snake_case.
- Python classes: PascalCase.
- Constants: UPPER_SNAKE_CASE.
- Use descriptive names; avoid names such as data1, temp2, final_code, or test123.

5. Mandatory Evaluator Run Contract

The evaluator must be able to clone the repository and start the application without inspecting the source code for custom setup instructions.

Mandatory primary command:

```
./run.sh
```

The script must:

1. Create or reuse a local virtual environment.
2. Install dependencies from requirements.txt.
3. Initialize the SQLite database/tables when required.
4. Start the Flask API on 127.0.0.1:5000.

The README must also document the manual fallback:

```
pip install -r requirements.txt
python run.py
```

A local SQLite database file must not be committed. Starting from a fresh clone must create everything necessary to run the application.

6. Standard API Contract

Method	Endpoint	Purpose
GET	/health	Application health
POST	/api/v1/trips	Create a trip
GET	/api/v1/trips	List trips
GET	/api/v1/trips/<trip_id>	Get one trip
PUT	/api/v1/trips/<trip_id>	Update a trip
DELETE	/api/v1/trips/<trip_id>	Delete a trip
POST	/api/v1/trips/<trip_id>/travelers	Add traveler
DELETE	/api/v1/trips/<trip_id>/travelers/ <traveler_id>	Remove traveler
POST	/api/v1/trips/<trip_id>/expenses	Add expense
PATCH	/api/v1/trips/<trip_id>/status	Change trip status
GET	/api/v1/trips/<trip_id>/summary	Calculated trip summary

7. Minimum Data Contract

Trip creation request must support these fields:

```
{
  "destination": "Cox's Bazar",
  "start_date": "2026-10-20",
  "end_date": "2026-10-23",
  "budget": 30000,
  "max_travelers": 5
}
```

Traveler creation request:

```
{
  "name": "Ayesha Rahman",
  "email": "ayesha@example.com"
}
```

Expense creation request:

```
{
  "title": "Hotel",
  "amount": 12000
}
```

Status update request:

```
{  
  "status": "ONGOING"  
}
```

IDs may be integers or string identifiers. Response objects may contain additional fields, but the required fields and endpoint paths must remain recognizable and consistent.

8. Three-Day Challenge Roadmap

Day 1 - Foundation, Database, and Trip CRUD

- Create the repository and project environment.
- Create Flask application entry point and run.sh.
- Configure SQLite persistence and initialize tables automatically.
- Implement GET /health.
- Implement create, list, retrieve, update, and delete trip endpoints.
- Validate required fields, positive budget/capacity, and date order.
- Return JSON and appropriate HTTP status codes.
- Add initial README and meaningful Git commits.

Day 1 checkpoint: the evaluator can start the project, create a trip, restart the server, and retrieve the same trip.

Day 2 - Travelers, Expenses, and Summary

- Implement traveler add/remove operations.
- Prevent the same traveler from joining the same trip twice.
- Enforce maximum traveler capacity.
- Prevent a traveler from joining trips with overlapping date ranges.
- Implement expense creation.
- Reject non-positive expenses and expenses that exceed the remaining budget.
- Implement a summary endpoint that calculates traveler_count, available_seats, total_expense, and remaining_budget.

Day 3 - Lifecycle Rules, Edge Cases, and Quality

- Implement trip lifecycle transitions.
- Enforce restrictions based on trip status.
- Handle invalid IDs and malformed/invalid input consistently.
- Review boundary cases and hidden-test readiness.
- Complete README/API documentation.
- Verify a fresh-clone run using ./run.sh.
- Clean repository artifacts and produce final meaningful commits.

Required lifecycle:

```
PLANNED -> ONGOING -> COMPLETED  
PLANNED -> CANCELLED  
ONGOING -> CANCELLED
```

All other transitions must be rejected, including transitions out of COMPLETED or CANCELLED.

9. Business Rules

Rule	Requirement
BR-01	Trip end_date must be later than start_date.
BR-02	budget must be greater than zero.
BR-03	max_travelers must be greater than zero.

BR-04	The same traveler must not be added to the same trip twice. Email should be treated as the traveler identity for conflict checking.
BR-05	A trip must never contain more travelers than <code>max_travelers</code> .
BR-06	A traveler must not participate in two trips whose date ranges overlap. Back-to-back trips where the first trip ends before the second begins are allowed.
BR-07	Expense amount must be greater than zero.
BR-08	Total trip expenses must never exceed the trip budget. Spending exactly the remaining budget is valid.
BR-09	<code>max_travelers</code> must not be reduced below the current traveler count.
BR-10	Travelers may be added only while the trip is <code>PLANNED</code> .
BR-11	Expenses may be added only while the trip is <code>PLANNED</code> or <code>ONGOING</code> .
BR-12	<code>COMPLETED</code> trips cannot be edited, accept travelers/expenses, or transition to another status.
BR-13	<code>CANCELLED</code> trips cannot be edited, accept travelers/expenses, or transition to another status.
BR-14	Only the lifecycle transitions defined in this assignment are valid.

10. HTTP and Error Response Rules

- Use 200 OK for successful retrieval/update/delete operations where appropriate.
- Use 201 Created for successful resource creation.
- Use 400 Bad Request for invalid request data or validation failures.
- Use 404 Not Found for unknown resources.
- Use 409 Conflict for business conflicts such as duplicate traveler, full trip, overlapping trip, budget conflict, or invalid lifecycle transition.
- Do not return HTTP 200 for failed business operations.

Recommended error shape:

```
{
  "error": "TRIP_FULL",
  "message": "The trip has reached its maximum traveler capacity."
}
```

11. Assignment Rules and Constraints

- Use Python, Flask, and SQLite.
- Persistent application data must be stored in SQLite, not primarily in Python lists/dictionaries.
- Do not commit local database files, virtual environments, `__pycache__`, `.env` files, or IDE-specific generated files.
- Do not change the required API prefix or endpoint paths.
- All API responses must be JSON.
- Business rules must be enforced server-side.
- `requirements.txt` is mandatory.
- `run.py` and `run.sh` are mandatory.
- The project must run on port 5000.
- Do not require the evaluator to execute SQL manually or edit source/configuration to start the project.

- A frontend, authentication, Docker, cloud deployment, and external database are not required.
- Third-party packages are allowed when justified, but unnecessary framework complexity should be avoided.
- AI tools may assist learning, but each participant is responsible for understanding and explaining every submitted part of the solution.

12. README Requirements

- Project overview and problem statement.
- Prerequisites.
- Fresh-clone run instructions using `./run.sh`.
- Manual run instructions.
- API endpoint table.
- Example requests/responses.
- Business rules/assumptions.
- Project structure.
- How SQLite is initialized/stored.
- Known limitations, if any.

13. Acceptance Criteria

Criterion	Observable Result
AC-01	A fresh clone starts using <code>./run.sh</code> and listens on 127.0.0.1:5000.
AC-02	GET <code>/health</code> returns HTTP 200 and JSON with status set to ok.
AC-03	A valid trip can be created and retrieved.
AC-04	Created data survives an application restart, demonstrating SQLite persistence.
AC-05	Invalid date order, non-positive budget, and non-positive capacity are rejected.
AC-06	Duplicate traveler participation in the same trip is rejected.
AC-07	Capacity is enforced, including the exact-full boundary.
AC-08	Overlapping trips for the same traveler are rejected.
AC-09	An expense equal to the remaining budget succeeds; an expense above it fails.
AC-10	Trip summary calculations are correct.
AC-11	Only allowed lifecycle transitions succeed.
AC-12	Status-dependent traveler/expense/edit restrictions are enforced.
AC-13	Unknown resources return 404 and failed operations do not return 2xx.
AC-14	Repository naming, required files, README, <code>.gitignore</code> , and dependency declaration follow the specification.

14. Hidden Evaluation and Live Review

The submitted API may be tested with additional boundary and invalid-operation scenarios that are not explicitly demonstrated in this document. The goal is to evaluate whether the implementation follows the rules rather than only matching sample requests.

- Expect boundary-value tests, missing/unknown IDs, repeated requests, overlapping dates, exact-budget cases, and invalid state transitions.
- During review, the trainer may ask for a small change to the business rules.
- The participant may be asked to identify where the change belongs, implement it, and explain the reasoning.
- The participant should be able to explain the database model, request flow, validation strategy, and key business-rule implementation.

15. Submission Checklist

- Repository follows {firstname}_{lastname}_trip_planner_batch_12.
- Final solution is available on the main branch.
- run.sh works from a fresh clone.
- run.py, requirements.txt, README.md, and .gitignore are present.
- SQLite initializes without manual SQL steps.
- All required endpoints and business rules are implemented.
- Local database/venv/cache files are not committed.
- README contains complete run and API instructions.
- Repository contains meaningful Git history.

16. Suggested Project Shape (Not Mandatory)

A clean project may look similar to the following, but equivalent structures are acceptable:

```
trip_planner/
├── run.py
├── run.sh
├── requirements.txt
├── README.md
├── .gitignore
├── app/
│   ├── __init__.py
│   ├── models.py
│   ├── routes.py
│   └── services.py
└── instance/
    └── trip_planner.db  # generated locally; do not commit
```